

ADVANTAGES OF JAVASCRIPT

ES6

MODULES

ARROWS

ENHANCED
OBJECT LITERALS

CLASSES

BLOCK
SCOPING

PROMISES

Desarrollo de aplicaciones para Internet

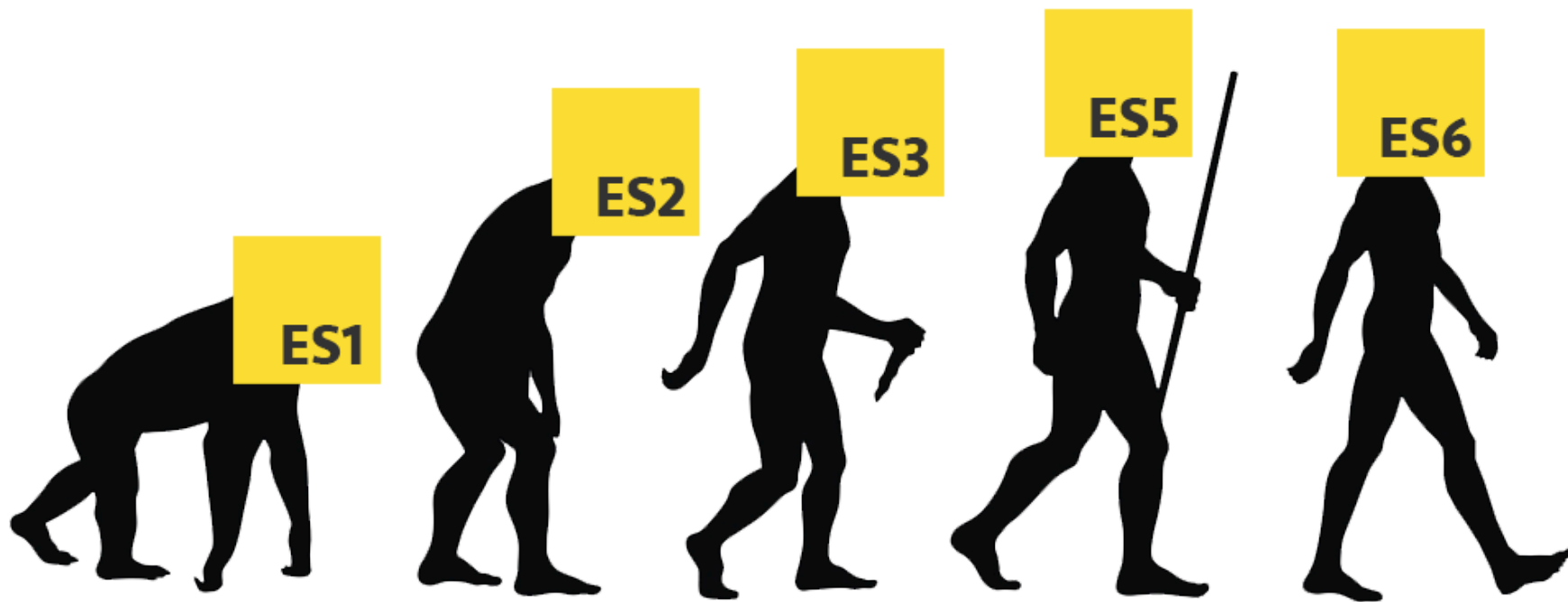
1997

1998

1999

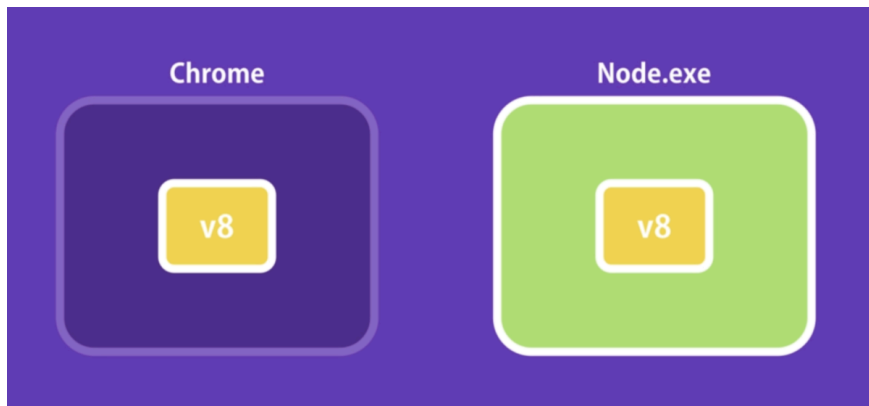
2009

2015



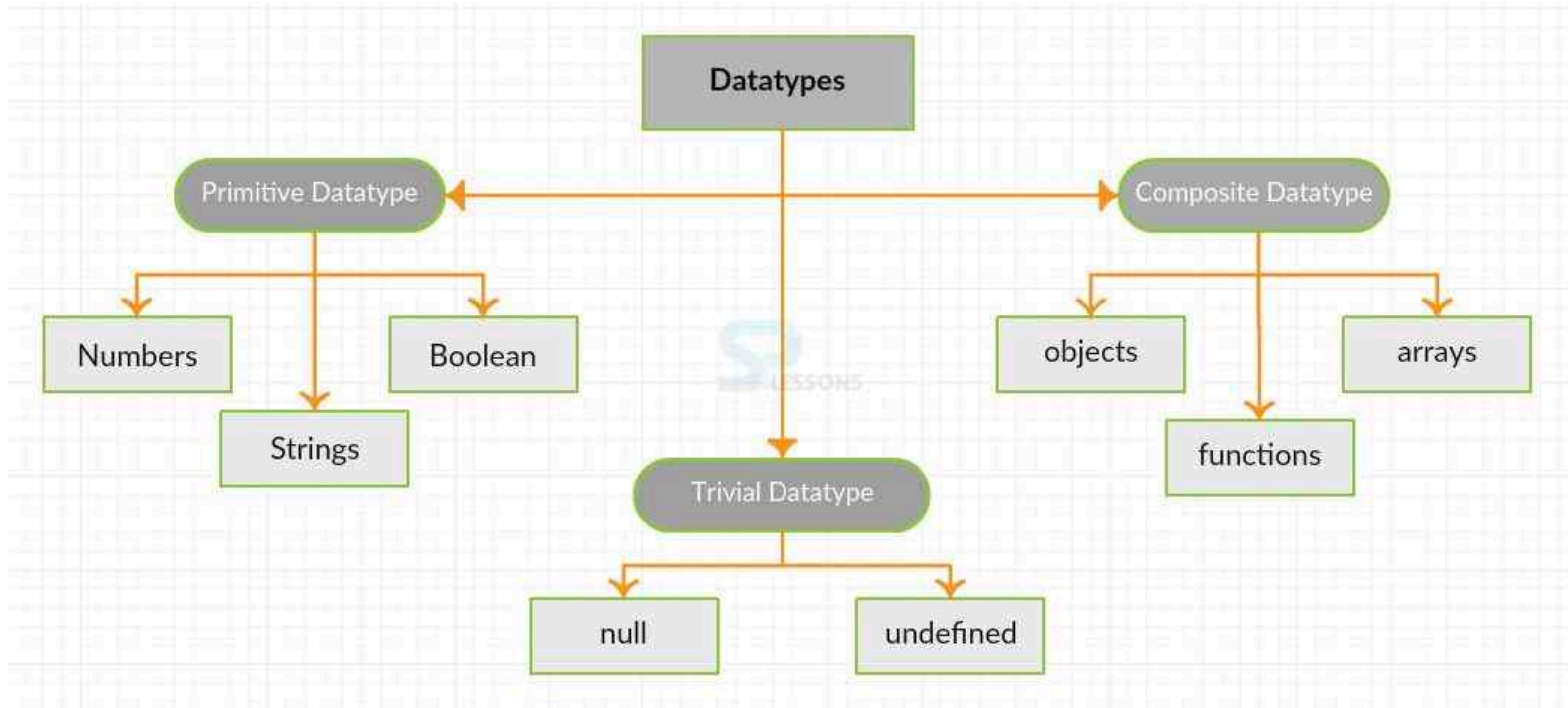
JS Evolution

JavaScript ES6



Learn JAVASCRIPT in just 5 MINUTES

Tipos



Data types

Declaración y alcance de las variables

```
// global
global.a = 7 // global scope visible en todos los módulos (node)
window.a = 7 // global scope en los navegadores

// local: bloque o módulo
let a = "hola"
a = 8 // 8, tipado dinámico

// como let pero error si se reasigna
const b = 6
b = 7 // error

{
  const b = 7 // sin error, scope bloque
}

var a = 8 // OBSOLETO NO USAR
```

Comparaciones

```
// A veces se cambia el tipo en las comparaciones  
// o hay sobrecarga de operadores
```

```
if ("") {} // false   conversión de tipo en los if  
if (0) {}  // false  
2 + "3"    // "23"    conversión tipo → concatenación
```

```
// igualdad de valor           =  
if (2 == "2") {} // true
```

```
// igualdad estricta de valor y tipo   ===  
if (2 === "2") {} // false
```



JavaScript Equality Operators EXPLAINED

Arrays

```
const frutas = ["Apple", "Orange", "Plum"]

frutas[0] // Apple
frutas.indexOf("Plum") // 2
frutas.includes("Mellon") // false

frutas.length // 3

frutas[1] = "Lemmon" // ["Apple", "Lemmon", "Plum"]

frutas.at(-1) // "Plum" valores negativos

frutas.push('Banana') // Añade al final
frutas.pop() // Quita al final

frutas.slice(0,1) // Devuelve ["Apple", "Lemmon"]
frutas.splice(1,2) // Desde el segundo elemento, elimina 2
```

Arrays

```
const frutas = ["Apple", "Orange", "Plum"]
```

```
for (const i=0; i<frutas.length; i++) {  
  console.log(frutas[i])  
}
```

```
// mejor  
for (let fruta of frutas) {           // con of  
  console.log(fruta)                 // los arrays son iterables  
}
```

```
// o con iteradores y funciones flecha  
fruta.forEach((fruta) ⇒ console.log(fruta))
```

```
fruta.map((fruta) ⇒ fruta.length)      // devuelve otro array [5, 7, 6]
```

```
fruta.filter((fruta) ⇒ fruta.length < 7) // devuelve ["Apple", "Plum"]
```

Arrays iterate

Objetos

```
const prop = "p"  
const x     = 3
```

```
const ob = {  
  a: 6, // propiedad  
  "pro piedad": 'hola',  
  x,  
  [prop+'q']: 4,  
  suma(a, b) {  
    return a + 2*b  
  }  
}
```

```
ob.a // acceso con punto  
ob['pro piedad'] // acceso con paréntesis  
ob.suma(3,4) // 11
```

Objetos

```
'a' in ob           // true

ob.z = 9           // son dinámicos, nueva propiedad
delete ob.a

for (let prop in ob) { // 'in' para los objetos
  alert( prop.ob );
}

Object.keys(ob)      // todas las propiedades
Object.values(ob)     // todas los valores

// elusión ES&
function makeUser(name, age) {
  return {
    name, // same as name: name
    age,  // same as age: age
  }
}
makeUser('Pepe', 12) // {name:'Pepe', age:12}
```

Maps and Sets

OBJECTS

VS.

MAPS

```
task = {  
  task: 'Code',  
  date: 'today',  
  repeat: true  
};
```

- 👉 More “traditional” key/value store (“abused” objects)
- 👉 Easier to write and access values with `.` and `[]`
- 👉 Use when you need to include **functions** (methods)
- 👉 Use when working with JSON (can convert to map)

```
task = new Map([  
  ['task', 'Code'],  
  ['date', 'today'],  
  [false, 'Start coding!']  
]);
```

- 👉 Better performance
- 👉 Keys can have **any** data type
- 👉 Easy to iterate
- 👉 Easy to compute size
- 👉 Use when you simply need to map key to values
- 👉 Use when you need keys that are **not** strings

JavaScript Maps vs Objects

ARRAYS

VS.

SETS

```
tasks = ['Code', 'Eat', 'Code'];  
// ["Code", "Eat", "Code"]
```

- 👉 Use when you need **ordered** list of values (might contain duplicates)
- 👉 Use when you need to **manipulate** data

```
tasks = new Set(['Code', 'Eat', 'Code']);  
// {"Code", "Eat"}
```

- 👉 Use when you need to work with **unique** values
- 👉 Use when **high-performance** is *really* important
- 👉 Use to **remove duplicates** from arrays

ES6: Set Vs Array- What and When?

Deestructuración de objetos y arrays

```
const direccion = {  
  calle: 'Pedro Antonio',  
  ciudad: 'Granada',  
  dp: '18004'  
}
```

```
// provincia, valor por defecto  
const { calle, ciudad, dp, provincia="Granada" } = direccion  
console.log(calle) // 'Pedro Antonio'
```

```
// o con el operador ...rest  
const { calle, ...resto } = direccion  
resto.dp // 18004
```



```
// lo mismo para arrays  
const [a, b, ...c] = [1, 2, 3, 4]  
console.log(a) // 1  
console.log(c) // [3, 4]
```

Object destructuring

Operador spread ...

```
// también a la derecha de las asignaciones
```

```
const a = [1, 2]
```

```
const b = [4, 5]
```

```
const c = [...a, 3, ...b]    // 1, 2, 3, 4, 5
```

```
const d = {a:1, b:2}
```

```
const e = {c:3, d:4}
```

```
const f = {...d, ...e}
```

```
f.d // 4
```

```
// uso para número indeterminado de argumentos en un función
```

```
function sumAll(...args) { // args is the name for the array
```

```
  let sum = 0;
```

```
  for (let arg of args) sum += arg; // sin bloque, una sola expresión
```

```
  return sum;
```

```
}
```

Rest parameters and spread syntax

Referencia y clonación

```
const ob_1 = { a: 'b' }  
const ob_2 = ob_1      // copia la referencia
```



```
ob_1.a = 'c'
```

```
ob_2.a      // ??
```

```
// clonación
```

```
const ob_3 = { ... ob_1 }  
ob_1.a = 'd'
```

```
ob_3.a      // ??
```

```
// los objetos constantes pueden ser modificados, son referencias
```

```
const user = {  
  name: "John"  
};  
user.name = "Pete";    // sin error
```

Igual para los arrays

Object references and copying

Funciones

```
// declaración, argumento por defecto
function Saluda_a(x, saludo="Hola") {
  console.log( `${saludo} ${x}!` );           // Templates ES6, comillas hacia atrás,
}
```

```
// Ejecución
Saluda_a('Peter') // Hola Peter
```

```
// Las funciones como los demás tipos son objetos, se pueden guardar en variables
const saluda = Saluda_a
saluda('Jhon')
```

```
// FUNCIONES FLECHA
const Saluda = (x) => {
  console.log( `Hello ${x}!` );
}
Saluda('Peter') // Hello, Peter
```

Getting Started with ES6 Arrow Functions in JavaScript

Functions

Clases

```
class User {  
  
    constructor(name) {  
        this.name = name;  
    }  
  
    edad = 0 // propiedad  
  
    Saluda() { // método  
        alert(`hola, soy ${this.name} tengo ${this.edad}`)  
    }  
  
}  
  
const user = new User("John") // clase mayúscula,  
user.edad = 24  
user.Saluda() // objeto minúscula por convención
```

Class

Módulos ES6

Cada archivo con código es un módulo

```
// 📁 sayHi.js
export function sayHi(user) {
  console.log(`Hello, ${user}!`)
}
```

```
export default function diHola(user) {
  console.log(`Hola, ${user}!`)
}
```

```
////////////////////////////////////
```

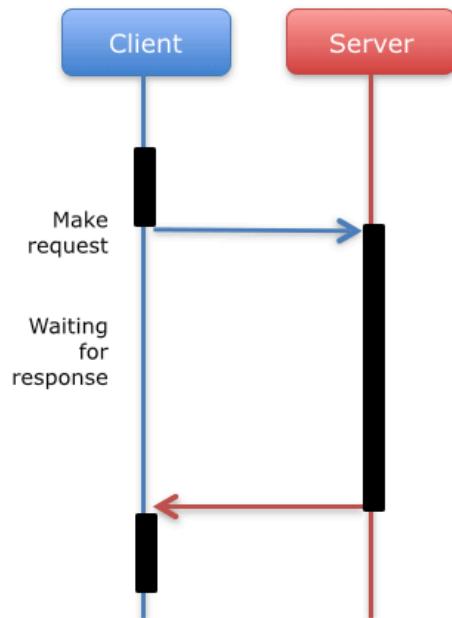
```
// 📁 main.js
import {sayHi} from './sayHi.js'
import diHola from './sayHay.js' // es la exportación por defecto
sayHi('Peter')
diHola('Pepe')
```

CommonJS vs. ES Modules in JavaScript

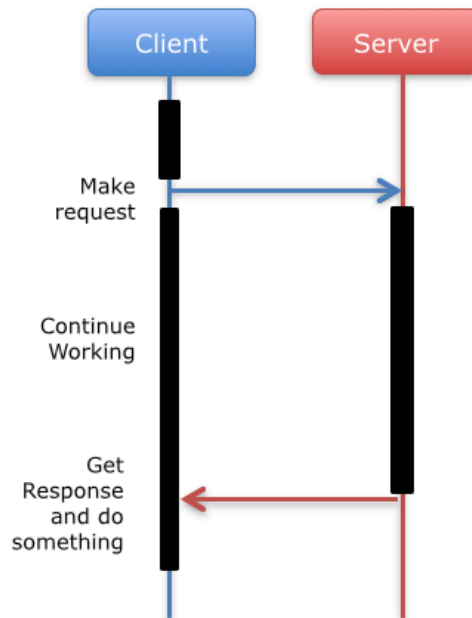
!Importante: Aclaraciones sobre el sistema de módulos de JavaScript

Modules

Synchronous



Asynchronous



Tradicionalmente con Callbacks

síncrono / asíncrono en NodeJS

```
import { unlinkSync } from 'node:fs'; // SÍNCRONO
```

```
try {  
  unlinkSync('/tmp/hello');  
  console.log('successfully deleted /tmp/hello');  
} catch (err) {  
  // handle the error  
}
```

```
import { unlink } from 'node:fs'; // ASÍNCRONO
```

```
unlink('/tmp/hello', (err) => { // con callback  
  if (err) throw err;  
  console.log('successfully deleted /tmp/hello');  
});
```

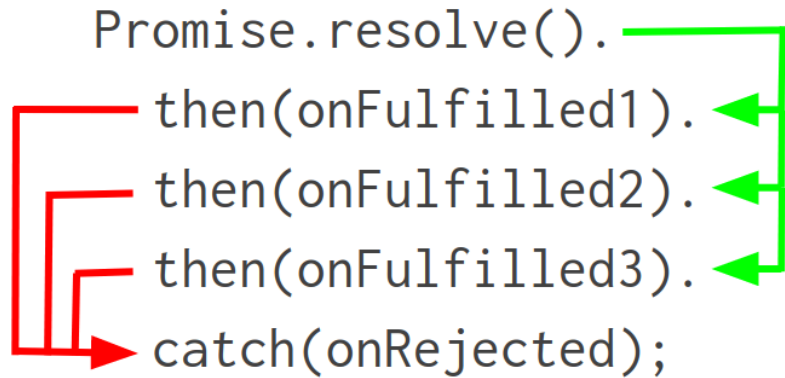
What is a Callback Function in JavaScript?

Problema: Callback Hell , si se anidan mucho

```
fs.readdir(source, function (err, files) {
  if (err) {
    console.log('Error finding files: ' + err)
  } else {
    files.forEach(function (filename, fileIndex) {
      console.log(filename)
      gm(source + filename).size(function (err, values) {
        if (err) {
          console.log('Error identifying file size: ' + err)
        } else {
          console.log(filename + ' : ' + values)
          aspect = (values.width / values.height)
          widths.forEach(function (width, widthIndex) {
            height = Math.round(width / aspect)
            console.log('resizing ' + filename + 'to ' + height + 'x' + height)
            this.resize(width, height).write(dest + 'w' + width + '_' + filename, function(err) {
              if (err) console.log('Error writing file: ' + err)
            })
          }).bind(this))
        }
      })
    })
  }
})
```

Mejor con **Promises ES6**, que se pueden encadenar

```
Promise.resolve().  
  then(onFulfilled1).  
  then(onFulfilled2).  
  then(onFulfilled3).  
  catch(onRejected);
```

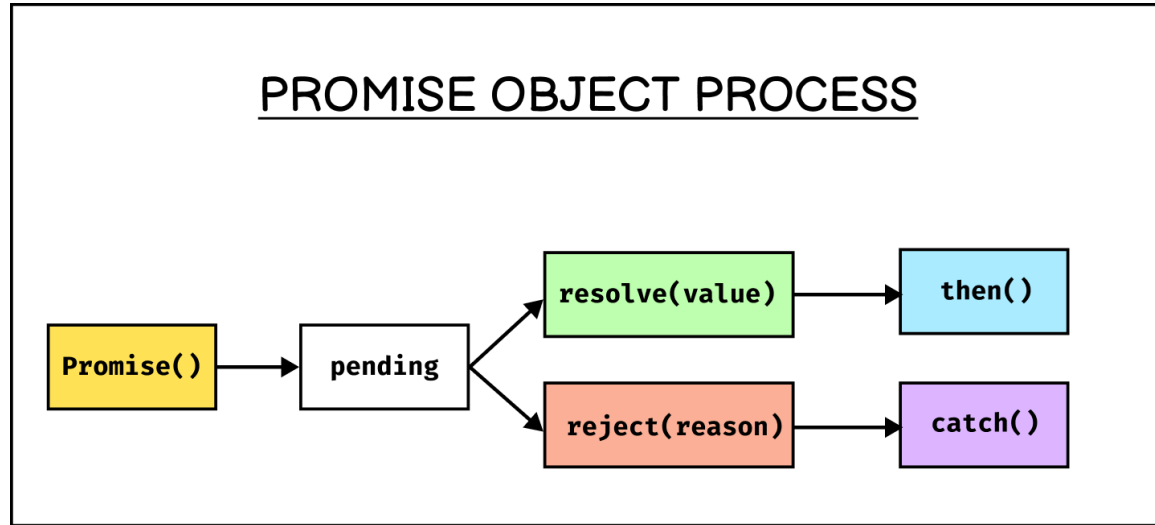
A diagram illustrating promise chaining. A green line starts from the right of 'Promise.resolve()' and branches into three green arrows pointing left to the right side of 'then(onFulfilled1)', 'then(onFulfilled2)', and 'then(onFulfilled3)'. A red line starts from the right of 'then(onFulfilled3)' and points right to the right side of 'catch(onRejected)'.

JavaScript Promise Tutorial: Resolve, Reject, and Chaining in JS and ES6 , Async/await

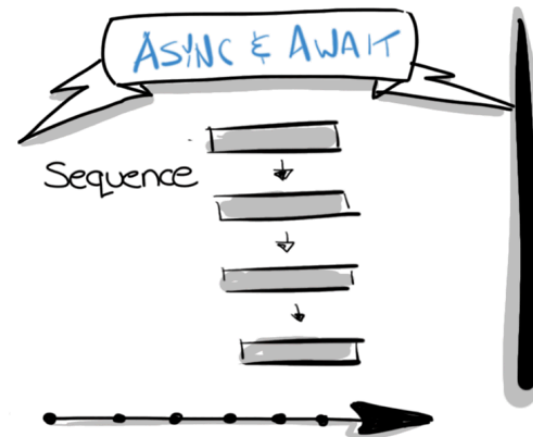
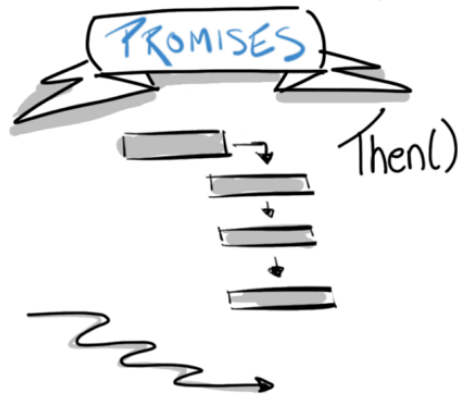
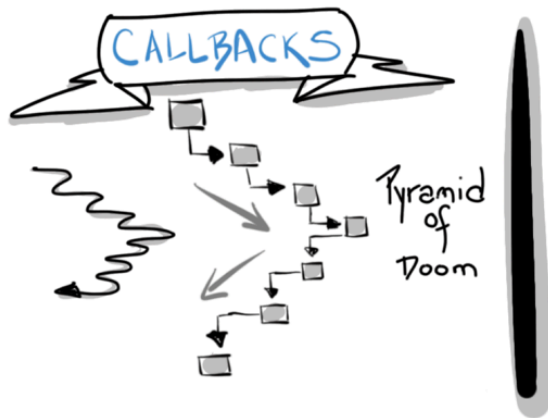


Javascript Promises vs Async Await EXPLAINED (in 5 minutes)

Promises



notes ES6 promises



Async/Await In 90 Seconds

Mastering JavaScript Promises: From Basics to Advanced

Top 10 de las mejores características de ES6

The Modern JavaScript Tutorial



JavaScript in 100 Seconds
